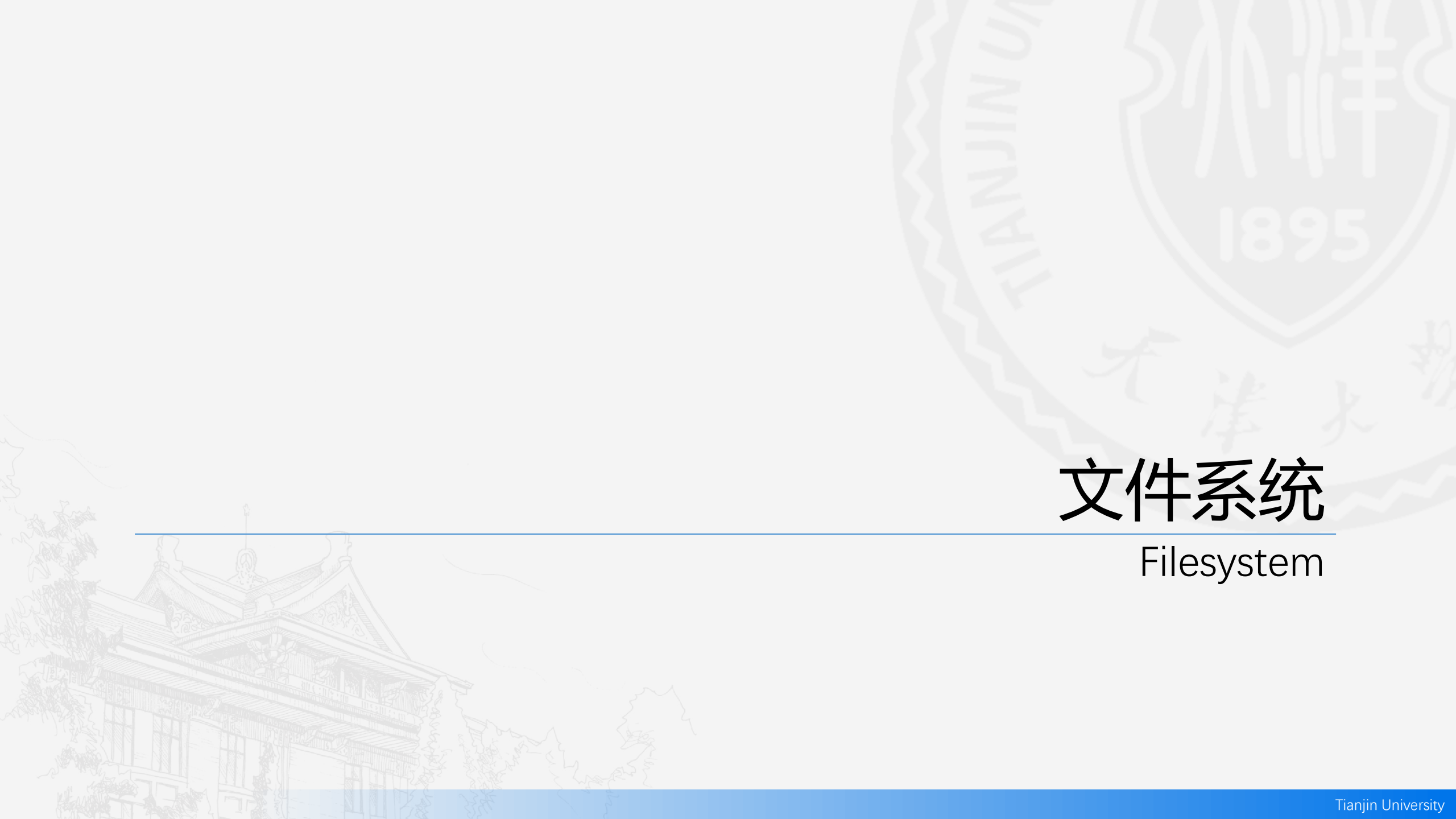


The background of the slide features a large, faint, light-blue circular logo of Tianjin University in the upper right corner. The logo contains the university's name in English ('TIANJIN UNIVERSITY') and Chinese ('天津大学'), along with the founding year '1895'. In the lower left corner, there is a faint, light-blue line drawing of a traditional Chinese building with a tiled roof and a flagpole on top.

文件系统

Filesystem



主要内容

- 文件
- 目录
- 文件系统



什么是文件

- 文件是一组数据的集合。
- 从操作系统的角度来看，文件是一个“字符流”或者“字节流”，文件没有内部结构。

A_collection_of_data\n

A_stream_of_characters_or_a_"byte_stream"\n

No_structure_is_imposed_on_a_file_by_the_operating_system\n

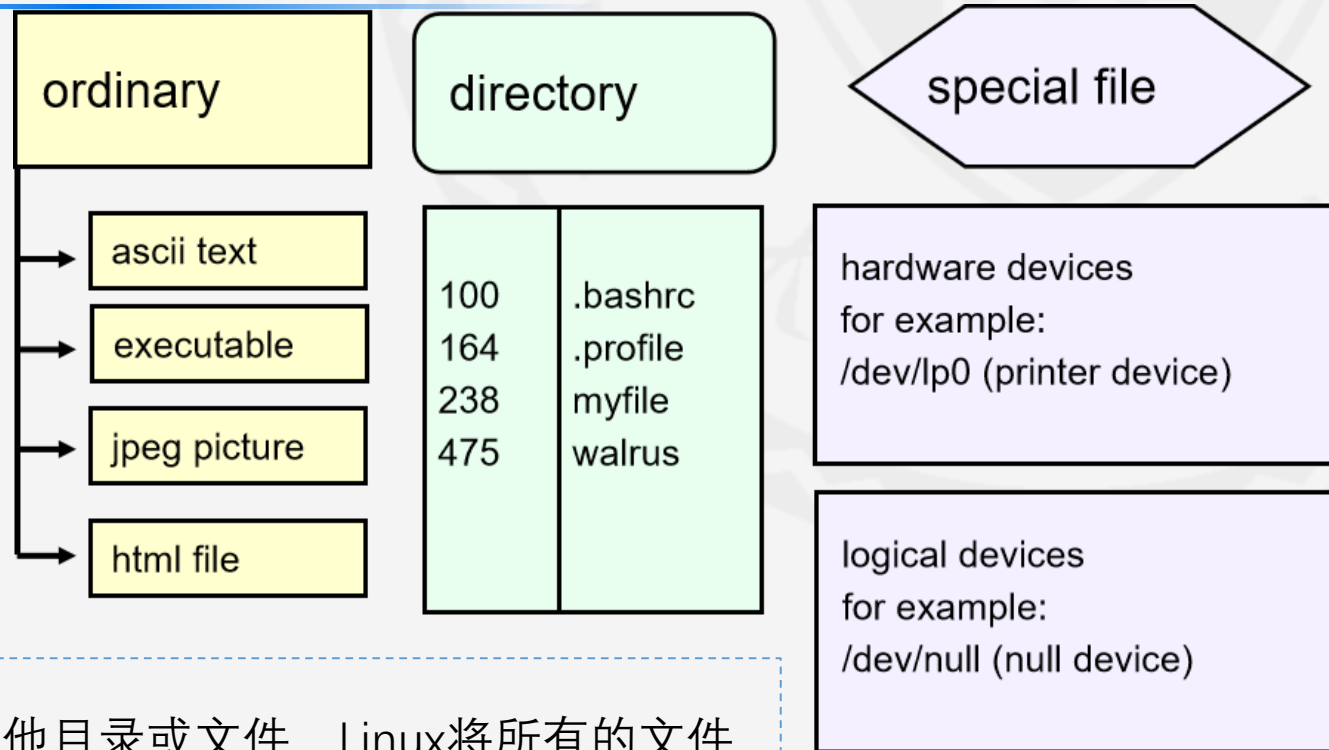
\n = newline character
_ = space character

A
File



文件类型

- 在Linux中，“文件”这个概念包含了三类概念，即普通文件、目录和特殊文件。以上三类均称为文件，即广义的“文件”。
- 普通文件即我们平时说的狭义的“文件”，往往是一组特定数据的集合，像一张图片，一首MP3，一部电影或一个文档等。



- 目录就是Windows中的“文件夹”，作用是存放其他目录或文件，Linux将所有的文件都存放在各个目录下，组成一棵目录树。
- 在Linux中，所有的设备都有文件与之对应，这些文件就属于特殊文件。



文件名

- 操作系统以“按名访问”的方式来访问文件。文件名应该反映文件的内容，并且要满足下列的条件。
- 文件名中不可以包含一些特殊符号。
 - Linux中不可以包含 路径分隔符/
 - Windows中不可以包含 盘符: 和 路径分隔符\
- 更重要的是，Linux的文件名是大小写敏感的，即ABC和abc是不同的二个文件，而在Windows中，二个文件名如果只有大小写上的差异，就被认为是同一个文件。
- 另外，Linux中文件名没有扩展名的概念，例如Linux中的可执行文件不一定要叫做*.exe。还有Linux中文件没有“隐藏”属性，在Linux中，如果一个文件的文件名的第一个字母是“.”，那么这个文件就是隐藏文件，在一般命令中是不查看隐藏文件的。

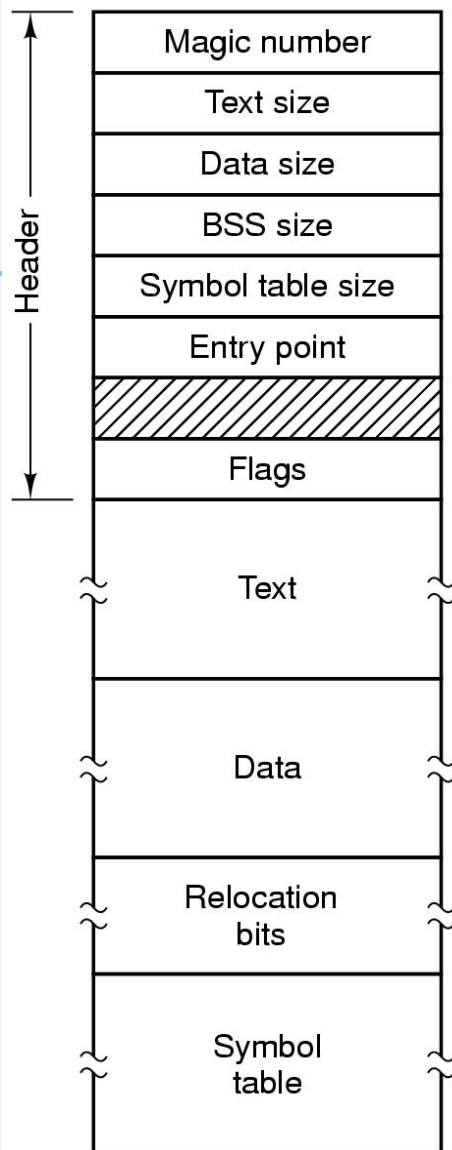


判断文件类型

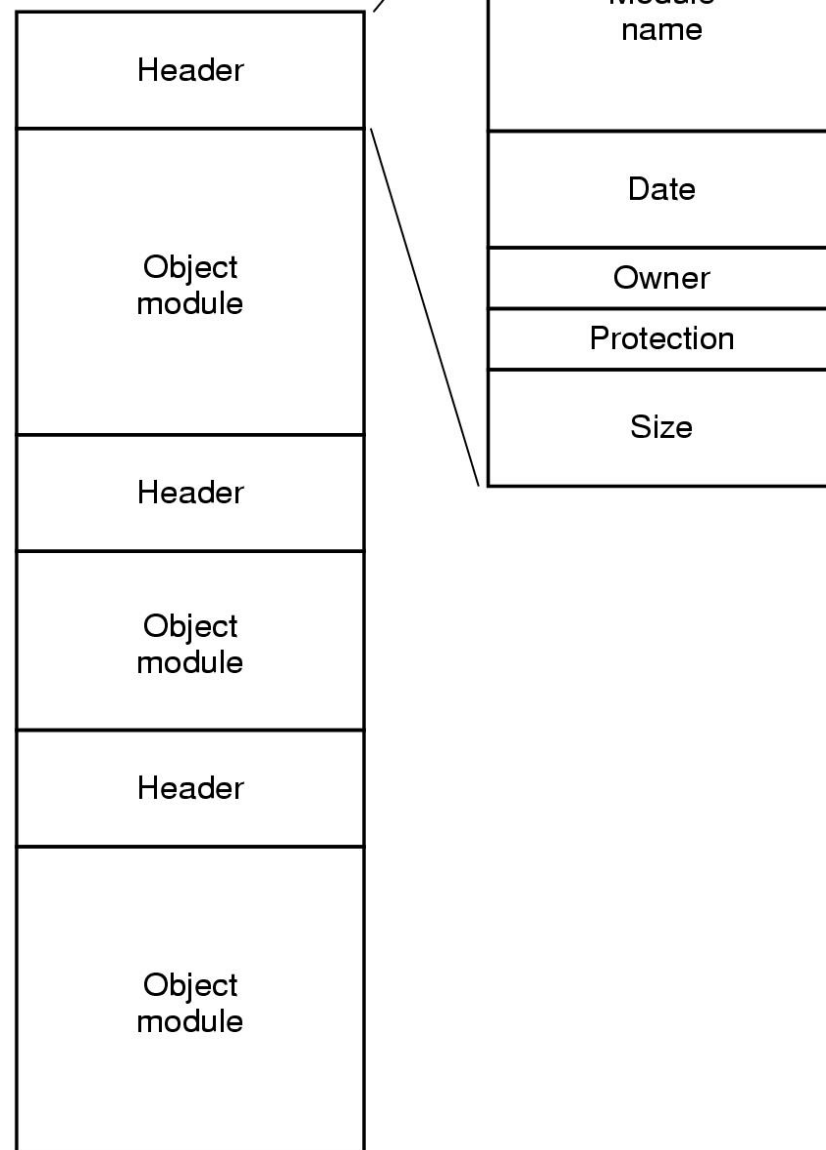
■ 操作系统判断一个文件的类型，并不仅是通过扩展名。文件开头的 Magic number 也是很重要的参考。例如zip文件总是以“PK”开头（zip算法发明人的名字缩写）

■ (a) An executable file.

■ (b) An archive.



(a)



(b)



文件属性

- 操作系统还会保存其他与文件相关的信息，如文件创建的日期和时间、文件大小等。
- 这些附加信息称为文件属性，也被称之为元数据。
- 文件的属性在不同系统中差异很大。

Attribute	Meaning
Protection	Who can access the file and in what way
Password	Password needed to access the file
Creator	ID of the person who created the file
Owner	Current owner
Read-only flag	0 for read/write; 1 for read only
Hidden flag	0 for normal; 1 for do not display in listings
System flag	0 for normal files; 1 for system file
Archive flag	0 for has been backed up; 1 for needs to be backed up
ASCII/binary flag	0 for ASCII file; 1 for binary file
Random access flag	0 for sequential access only; 1 for random access
Temporary flag	0 for normal; 1 for delete file on process exit
Lock flags	0 for unlocked; nonzero for locked
Record length	Number of bytes in a record
Key position	Offset of the key within each record
Key length	Number of bytes in the key field
Creation time	Date and time the file was created
Time of last access	Date and time the file was last accessed
Time of last change	Date and time the file was last changed
Current size	Number of bytes in the file
Maximum size	Number of bytes the file may grow to



文件属性 inode

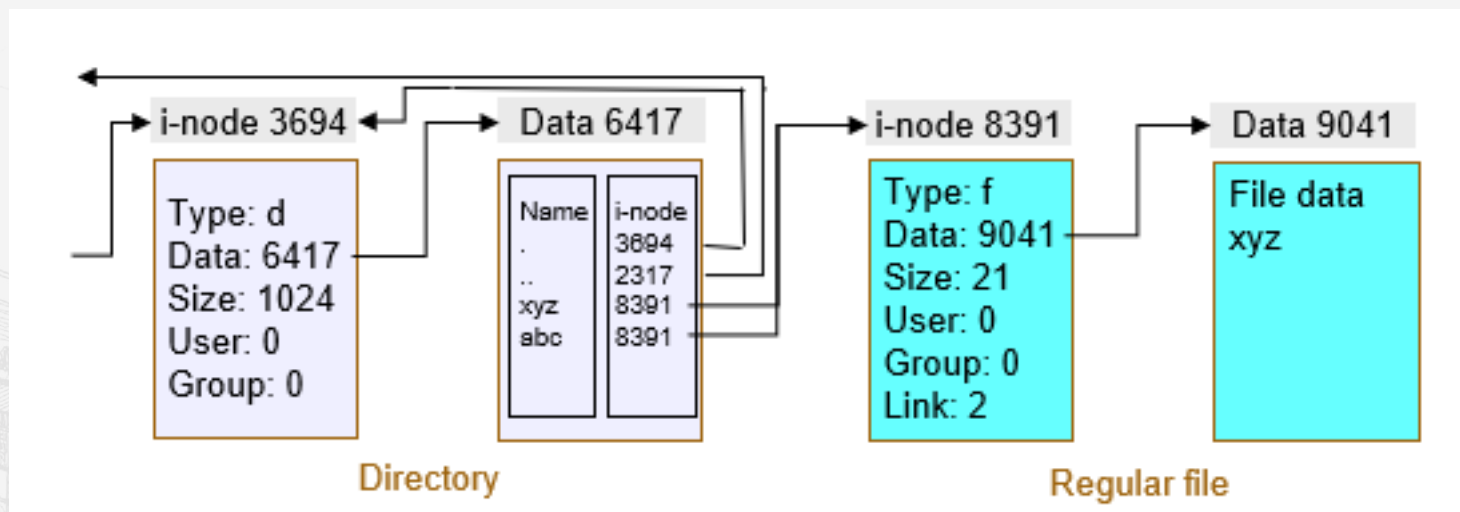
- Linux通常将文件属性存放在一种叫做 inode 的数据结构中。
- 每个文件有一个唯一的inode，每个inode也唯一的对应一个文件。
- 在一个文件系统中，每个inode有其唯一的编号。
- 每个inode保存了文件系统中的文件对象（包括文件、目录、设备文件、socket、管道,等等）的元信息数据，通常包括文件的所有人、所在组、文件类型、大小、权限、创建时间、最后访问时间和最后修改时间等，也包括指向文件数据区的指针，**但不包括文件名**。

Owner / Group
File Type
File Size
File Permissions
Time Stamps: create time access time modification time
Link Counter
Additional Flags: (ACL, EXT2, _FLAGS)
Pointers to Block Data



文件名 与 inode 如何建立关联

- inode中不存放文件名。文件名存放在所在的目录的数据中。
- 目录文件的数据就是一张表， 一列是文件名， 另一列是inode编号。
- 因此， 可以实现多个文件名指向同一个 inode， 即同一个文件有多个文件名。



例：

- 文件名 xyz 和 abc 都指向 inode 8391
- 或者说， inode 8391有二个文件名
- 不同目录下的文件名也可以指向同一个inode



```
ln source_file target_file
```

链接文件

- 如何为同一个文件建立多个文件名？文件名在Linux中的被称为“**链接**”（**Link**）。使用**ln**命令建立新链接。
- 注意rm删除的是链接，并不是文件本身。文件最后一个链接被删除时文件才真正被删除。
- 不可以为目录建立多个链接，因为可能形成环（例：目录a下有目录b，目录b下有目录c，而c就是a的别名）。但是操作系统自己可以这么干（例：每个目录下都有一个..目录是其父目录）
- 不可以跨文件系统为文件建立多个链接，因为不同文件系统中inode是分别编号的。

```
$ ls -li
63 -rw-r--r-- 1 team01 staff 1910 Nov 21 14:19 man_files

$ ln man_files manuals

$ ls -li
63 -rw-r--r-- 2 team01 staff 1910 Nov 21 14:19 man_files
63 -rw-r--r-- 2 team01 staff 1910 Nov 21 14:19 manuals
```

link counter

- link counter 表示文件的链接数量，可以看出 ln 后从1变成了2
- inode 63 本来只有一个链接（文件名）
- 经过 link 后，inode 63 有了二个链接（文件名）



```
ln -s source_file target_file
```

硬链接 vs 符号链接

- 链接有一个类似的概念叫**符号链接 (symbolic link)**，为区别也将链接称为**硬链接 (hard link)**
- 符号链接仅是对原文件名的间接引用，建立和删除对原文件不产生任何影响，包括link counter。
- 符号链接引用的是原文件名，不是inode，原文件名灭失后符号链接即为不可用状态。
- 可以对目录建立符号链接，也可以跨文件系统建立符号链接。

```
$ ls -li
63 -rw-r--r-- 1 team01 staff 1910 Nov 21 14:19 man_files

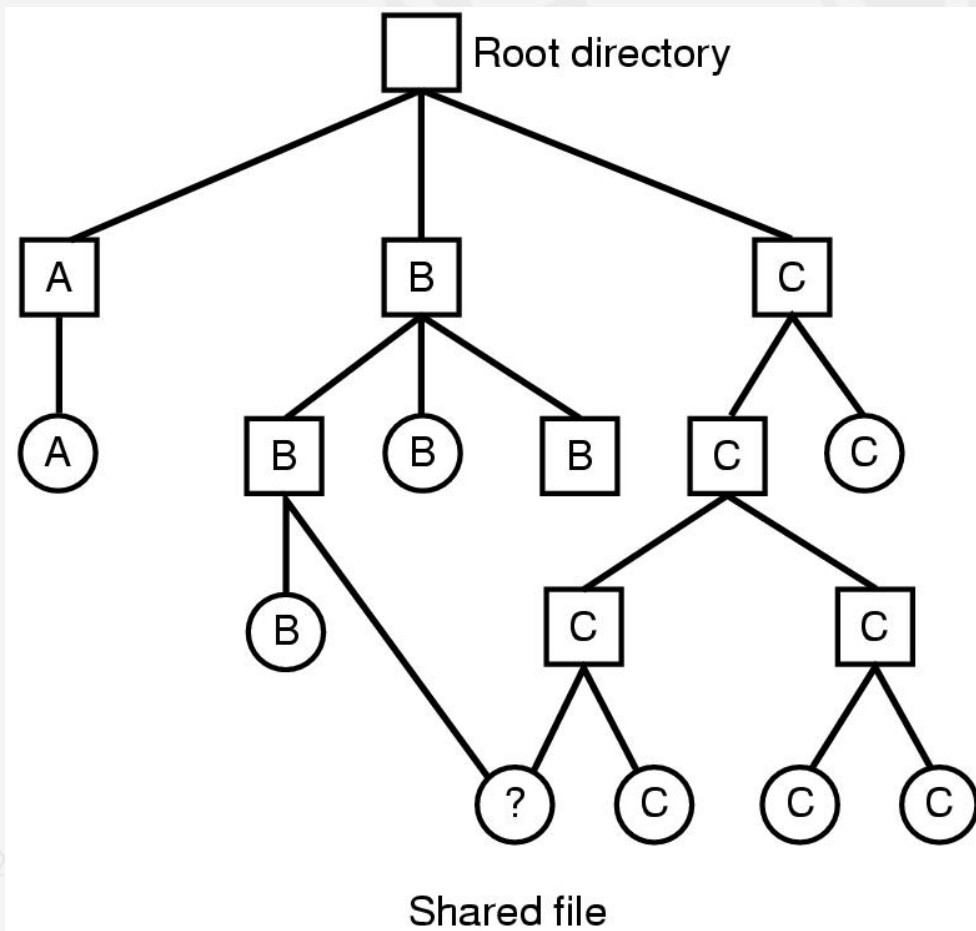
$ ln -s man_files manuals

$ ls -li
63 -rw-r--r-- 1 team01 staff 1910 Nov 21 14:19 man_files
66 lrwxrwxrwx 1 team01 staff 1910 Nov 21 14:19 manuals -> man_files
```



文件共享

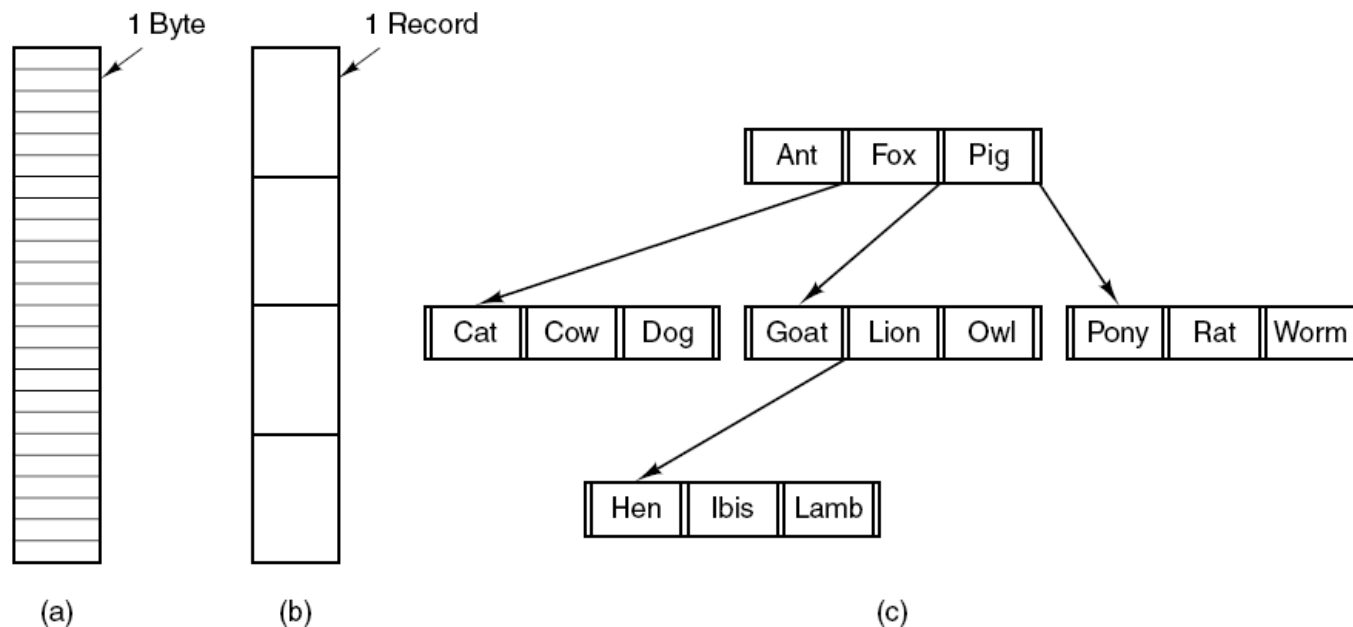
- 使用链接和符号链接，可以实现不同目录或者不同用户之间文件的共享。
- 例如：用户B和用户C可以分别在自己的目录中访问同一个共享文件。





文件的逻辑结构

- 文件的逻辑结构，指一个文件在用户面前所呈现的形式。又称文件组织。
- 虽然从操作系统角度看，文件没有内部结构，但是从应用程序角度看，文件有内部逻辑结构。
 - 字节流式结构
 - 记录式结构





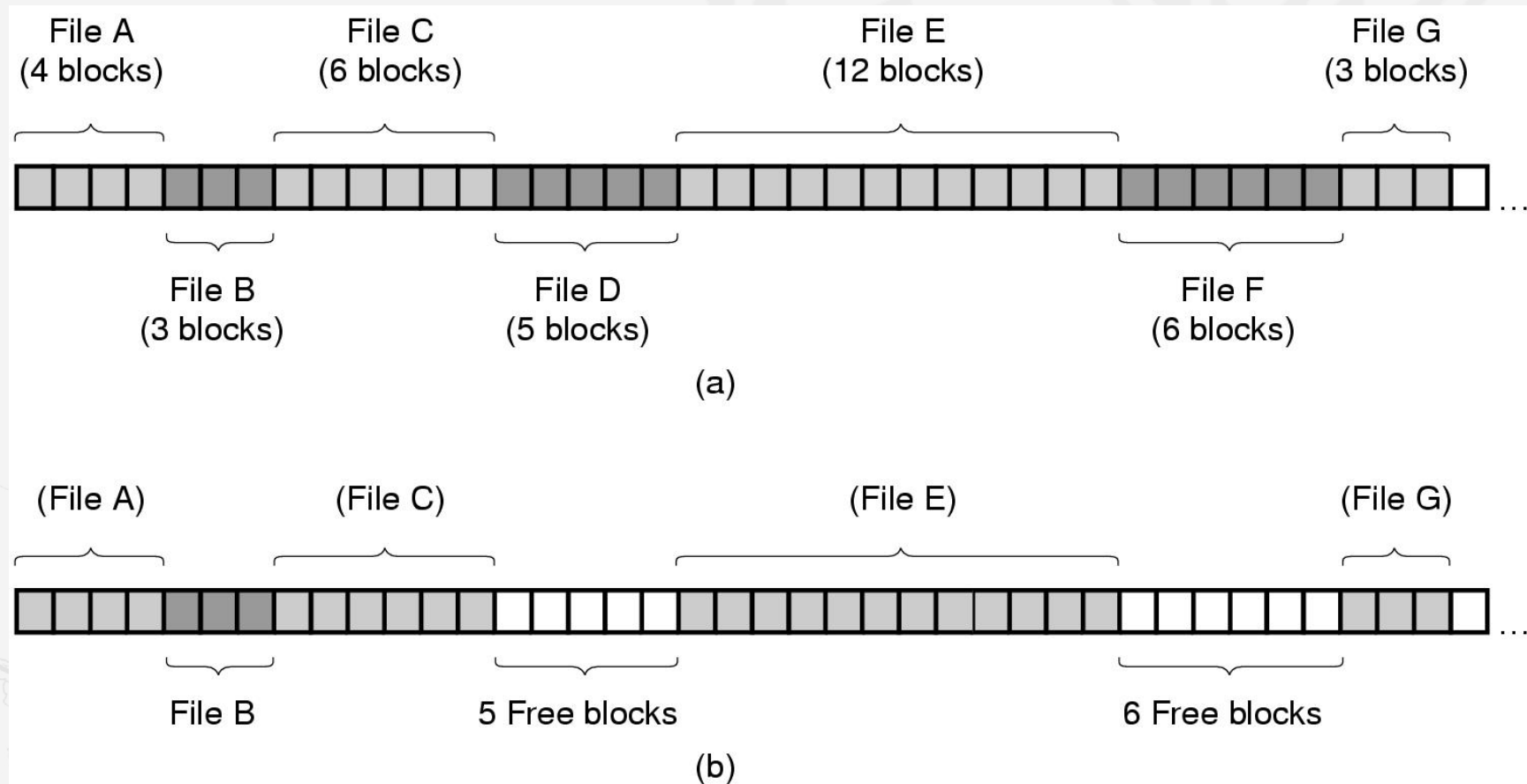
文件的物理结构

- 文件的物理结构，指文件在外存上的存储组织形式。这不仅和存储介质的存储性能有关，还与所采用的外存分配方式有关。
- 磁带是一种顺序存取设备，对磁带上的用户文件信息只能顺序访问。故磁带文件的物理结构也只能是将文件连续地存放在磁带上（分配方式），即磁带文件的物理结构只有一种顺序结构。
- 磁盘上的文件，既可以顺序存取，也可以随机（直接）存取。磁盘常用的外存分配方法有连续分配、链接分配和索引分配三种。采用不同的分配方式时，将形成不同的文件物理结构。
- 大容量存储设备管理空间通常以数据块（也称为簇）为单位管理。



连续物理结构

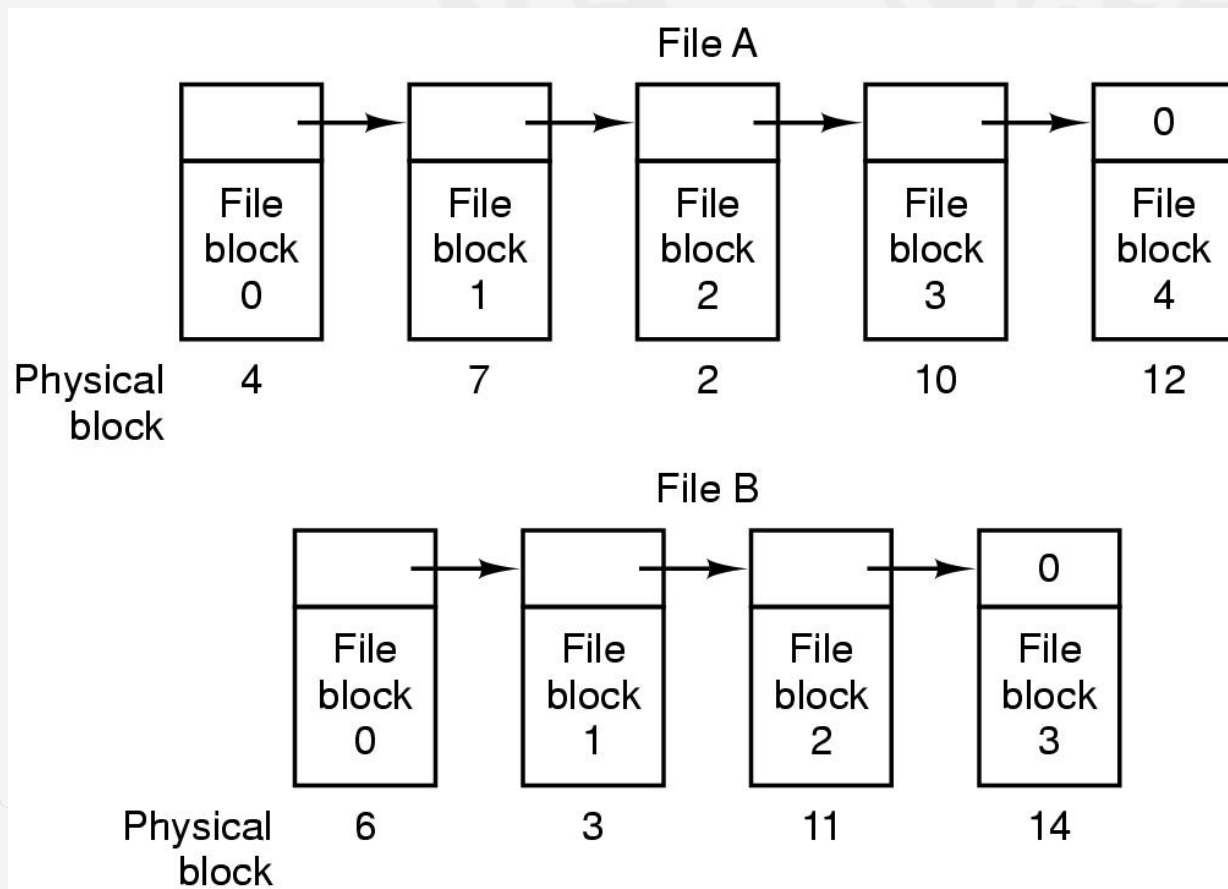
- 连续分配要求一个文件的数据块连续存放。这有利于随机访问，但是增大或减小文件很困难。
- 文件D和F删除后，形成了（外部）碎片，很难再被利用。





链接物理结构

- 链接分配可以充分利用每个磁盘块，不会因为磁盘碎片（除了最后一块中的内部碎片）而浪费存储空间。
- 链接分配有利于文件增大或减小，但是不利于随机访问。
- 文件属性中需要有一个指针指向数据块链的第一个块。



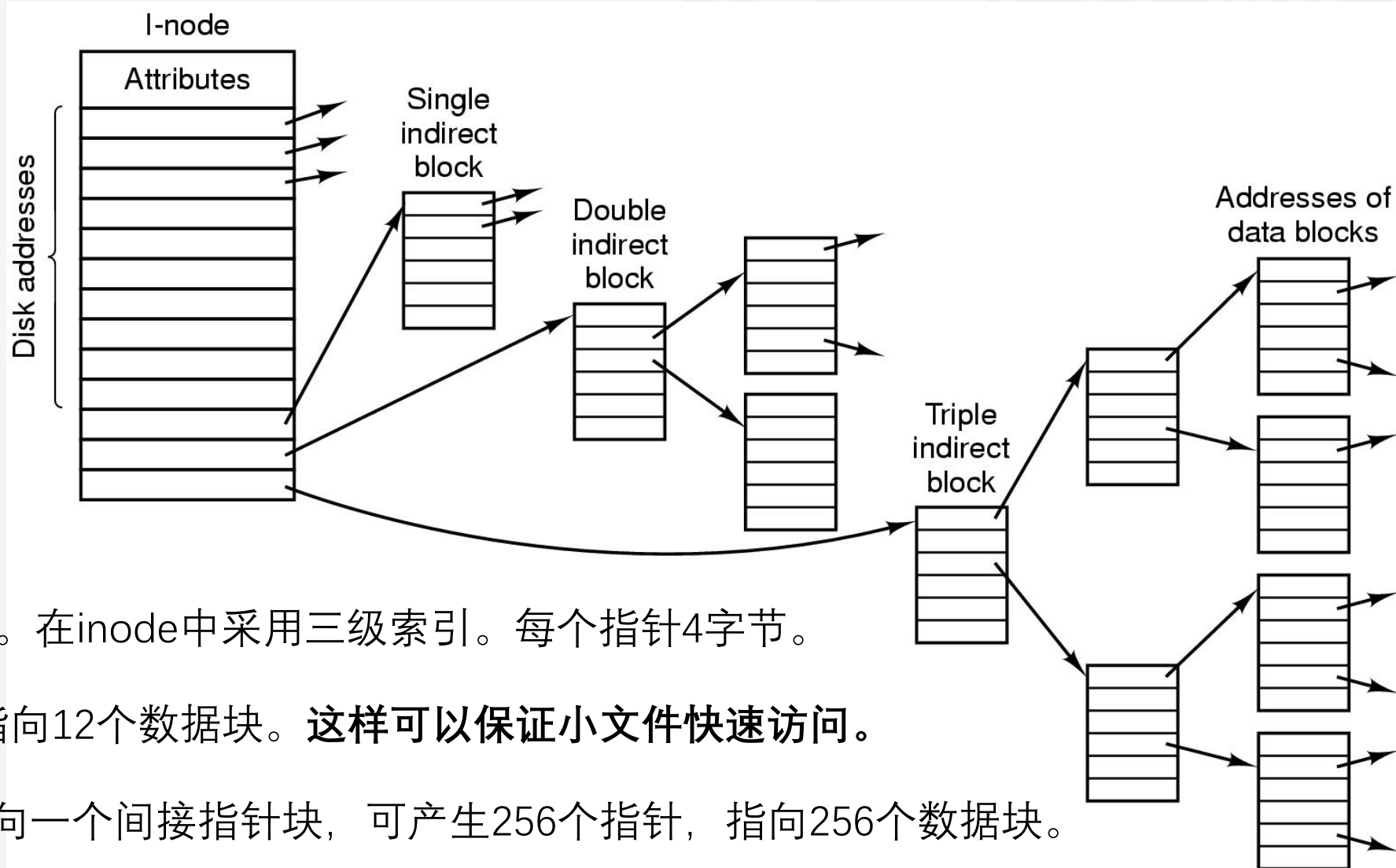


文件系统

Filesystem

索引物理结构

Linux采用多级索引结构来管理文件数据块。



- 例：数据块大小为1KB。在inode中采用三级索引。每个指针4字节。
- 直接指针12个，可以指向12个数据块。这样可以保证小文件快速访问。
- 一级间接指针1个，指向一个间接指针块，可产生256个指针，指向256个数据块。
- 二级间接指针1个，指向二级间接指针块，可产生 256^2 个指针，指向64K个数据块。
- 三级间接指针1个，指向三级间接指针块，可产生 256^3 个指针，指向16M个数据块。



例题

■ 某文件系统索引节点 (inode) 中包含13个直接块指针、1个一级间接块指针和1个二级间接块指针，间接块指向的是一个索引块，每个索引块和数据块的大小一致，均为1KB，地址指针所占空间为4B。

1. 2。仅计算硬链接的个数。
2. 提示文件不存在。
3. 3次。直接指针能访问13KB，一级指针能访问到256KB，
4. 所以1MB的内容，需要访问二级指针，1级+2级+数据，共三次访问硬盘
5. $(13+(1024/4)+(1024/4)^2)*1024 \approx 64\text{MB}$
6. $(13+(4096/4)+(4096/4)^2)*4096 \approx 4\text{GB}$
7. $(13+(1024/4)+(1024/4)^2+(1024/4)^3)*1024 \approx 16\text{GB}$

- 1、若某inode共有2个硬链接 (hard link)，分别为a和b，另有1个符号链接 (symbolic link) x->a，则该inode的link counter为_____。
- 2、将a删除后，访问x，结果为_____。
- 3、假设该索引节点已经被加载进内存中，则若要读取文件的第1MB的内容，需要访问磁盘_____次。
- 4、该文件系统能支持的文件最大容量约为_____。
- 5、若将数据块的大小修改为4KB，则该文件系统能支持的文件最大容量约为_____。
- 6、若保持数据块大小1KB不变，在不增加inode中的指针个数的前提下，取消一个直接块指针，增加一个三级间接块指针，则能支持的文件最大容量约为_____。



文件系统

- **文件系统 (File system)** 是一种存储和组织持久数据的方法，它使得对数据访问和查找变得容易，文件系统使用文件和树形目录的抽象逻辑概念代替了硬盘和光盘等物理设备使用数据块的概念，用户使用文件系统来保存数据不必关心数据实际保存在硬盘（或者光盘）的地址为多少的数据块上，只需要记住这个文件的所属目录和文件名。
- 文件系统的基本数据单位是文件，它的目的是对磁盘上的文件进行组织管理。一个文件系统通常包含一个树形的目录结构，目录树有唯一的一个根目录，根目录下面有一些节点，称为中间节点，最终的文件是叶子节点。
- 文件系统可以存储在任何**块设备 (block device)** 内，如软盘、硬盘、硬盘分区、光盘、U盘等，也包括RAID、逻辑卷、loop文件（如光盘镜像文件）甚至是内存虚拟硬盘（ram disk）中。



文件系统

- 文件系统的目的是对磁盘上的文件进行组织管理，组织的方式不同，就会形成不同的文件系统。
 - UNIX / Linux: minix, ext2, ext3, ext4, ReiserFS, IBM JFS, xfs
 - Windows: FAT-12, FAT-16, FAT-32, exFat, NTFS
 - Mac: HFS, HFS+, HFSX
 - CD/DVD: CDFS(ISO 9660), UDF(Universal Disk Format)
 - Network : NFS (Network File System), SMBFS (Windows share)
 - Virtual: SHMFS (Shared Memory Filesystem)
-

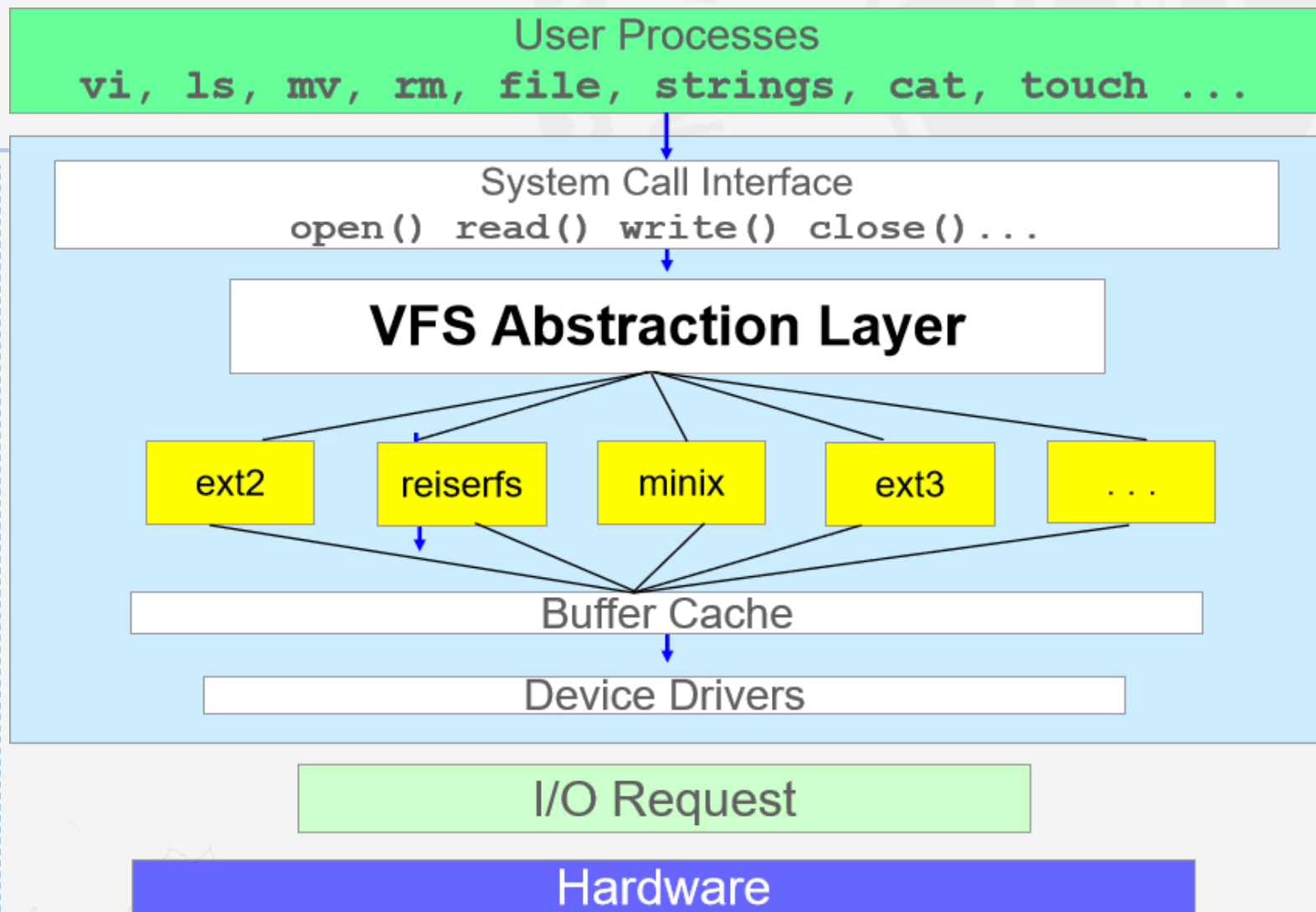


文件系统

Filesystem

虚拟文件系统

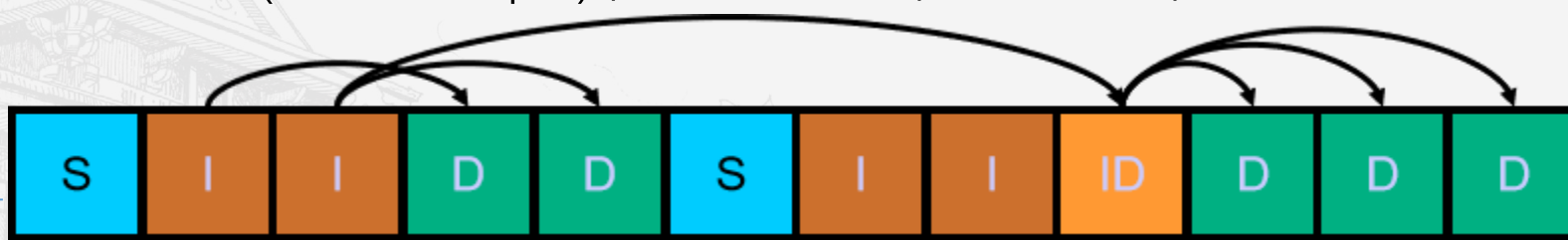
- 虚拟文件系统 (VFS) 是一个在具体文件系统之上的抽象层, 用于处理与POSIX文件系统相关的所有调用。它为各种文件系统提供了一个通用的接口, 使上层应用程序能够使用统一的接口访问不同文件系统, 同时也为不同文件系统的通信提供了媒介。





例：ext2 文件系统

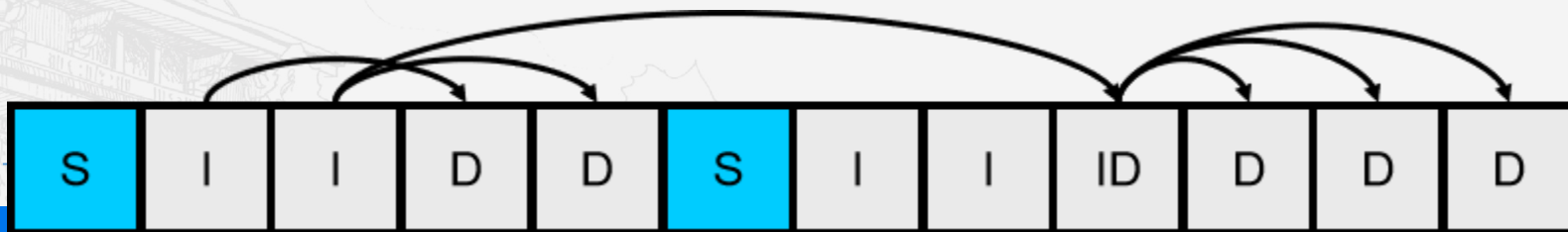
- ext2是Linux上的第一个商业级文件系统，与其他Unix中的文件系统具有相同的设计标准。
- 文件系统在创建时，被分成若干大小相同的数据块。数据块的大小的选择依赖于文件系统的大小和实际需求，可能是1KB、2KB、4KB等。数据块选择过小，则能支持的单一文件大小的上限就较小（inode多重索引决定单一文件能支持的数据块个数上限）。数据块选择过大，则文件浪费空间较大（每个文件占据整数个数据块，一般来说，最后一个数据块可能装不满）。
- 数据块有四种用途：超级块（Superblock）、索引结点块（Index node (inode) block）、间接指针块（Indirect block (double, triple)）和数据块（Data block）。





ext2的超级块

- 超级块 (Superblock) 是文件系统中的第1块，包含了文件系统的关键数据，因此非常重要，有若干备份 (第 8193, 16385, ... 块)。其中的内容包括：
 - 最后使用时间 Last mounted time/place
 - 数据块大小 Block size
 - 指向空闲inode块指针 Pointers to free inodes
 - 指向空闲数据块指针 Pointers to free blocks
 - 指向文件系统根目录的指针 Pointer to root of filesystem

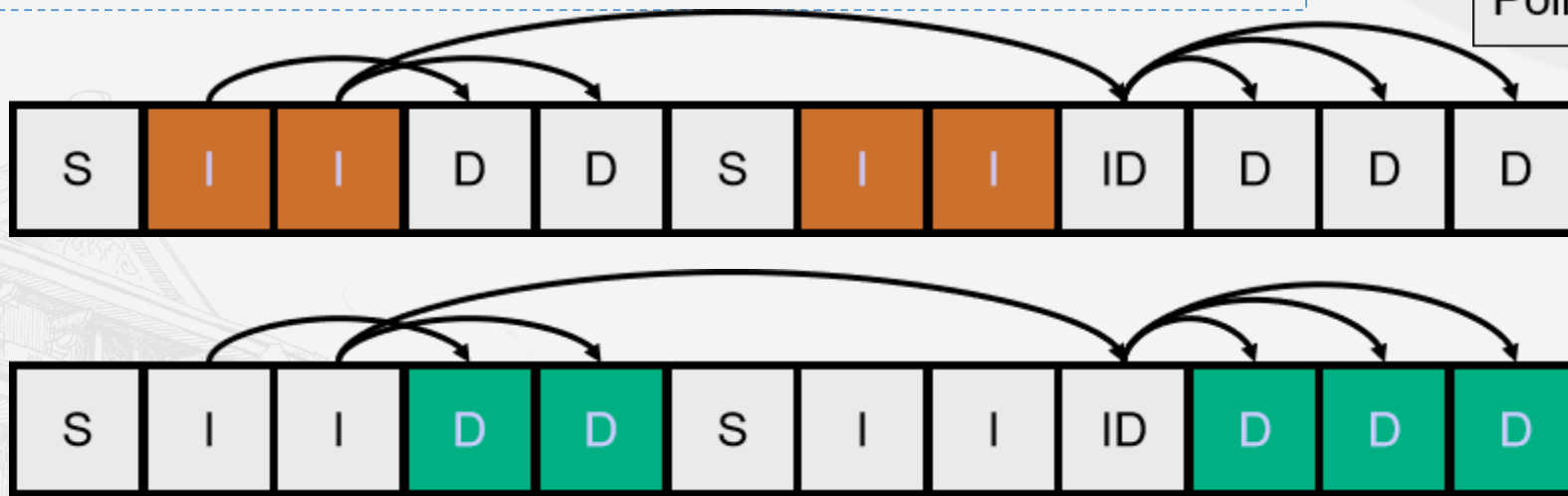




ext2的索引结点块和数据块

- ext2文件系统中的inode大小为128字节，如果数据块大小取1KB，则每个索引结点块中可以存入8个inode。
- 每个inode包含15个指针，分别指向存放文件数据的数据块或间接指针块。

Owner / Group
File Type
File Size
File Permissions
Time Stamps: create time access time modification time
Link Counter
Additional Flags: (ACL, EXT2_FLAGS)
Pointers to Block Data



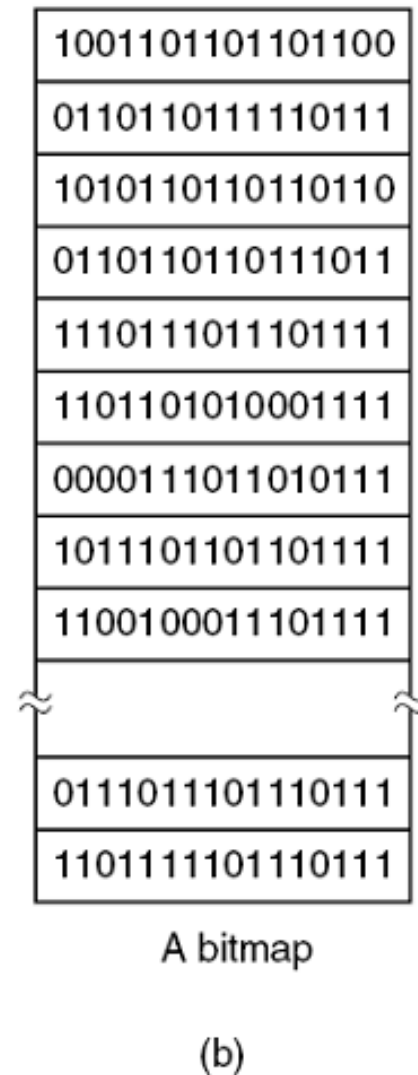
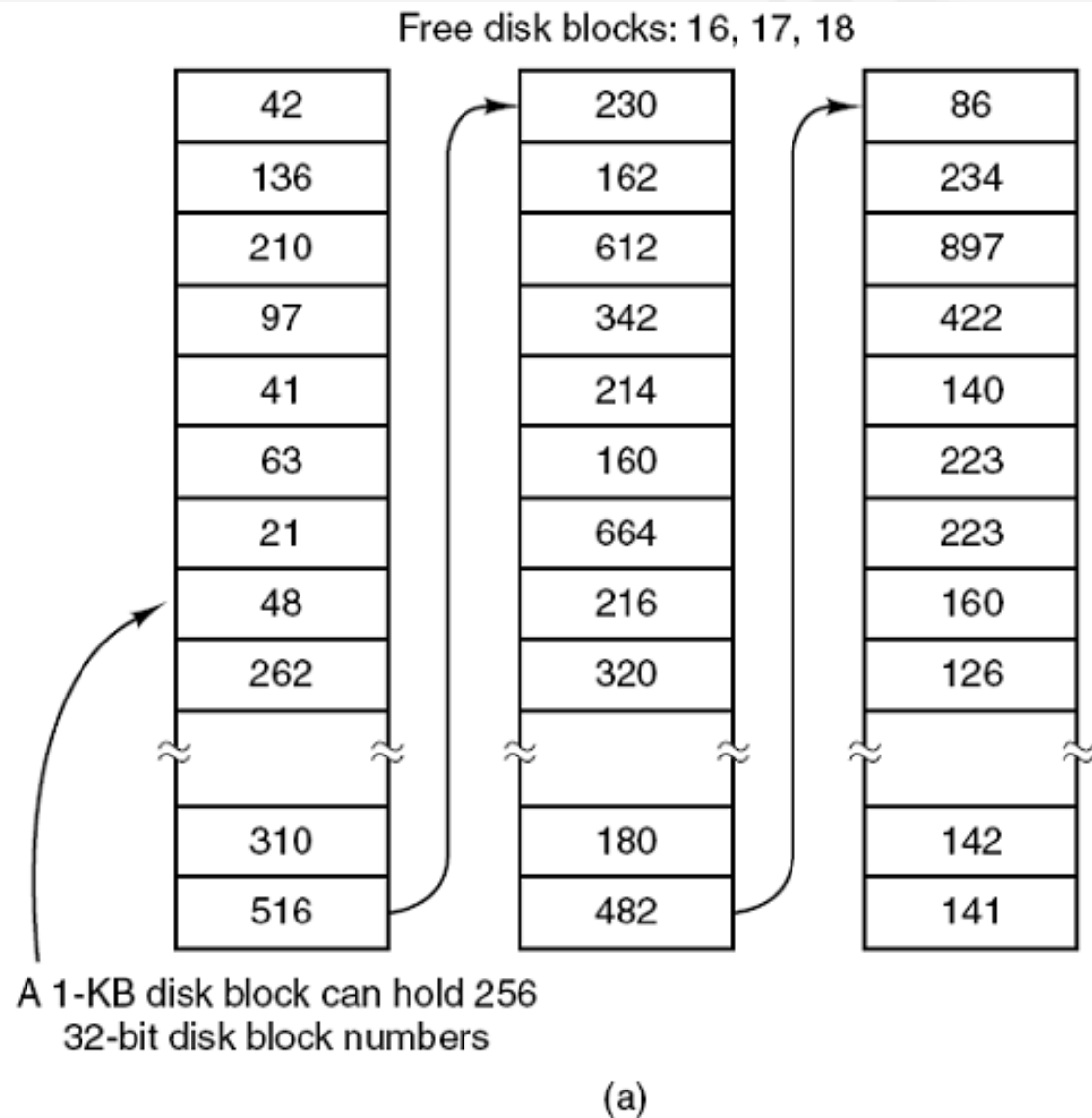


文件系统

Filesystem

管理空闲块

- 磁盘空闲块链表：磁盘块中存储空闲磁盘块号，最后一个存储指向下一个磁盘块的指针
- 位示图：每一个空闲块用一位表示，空闲块用1表示，已分配块用0表示

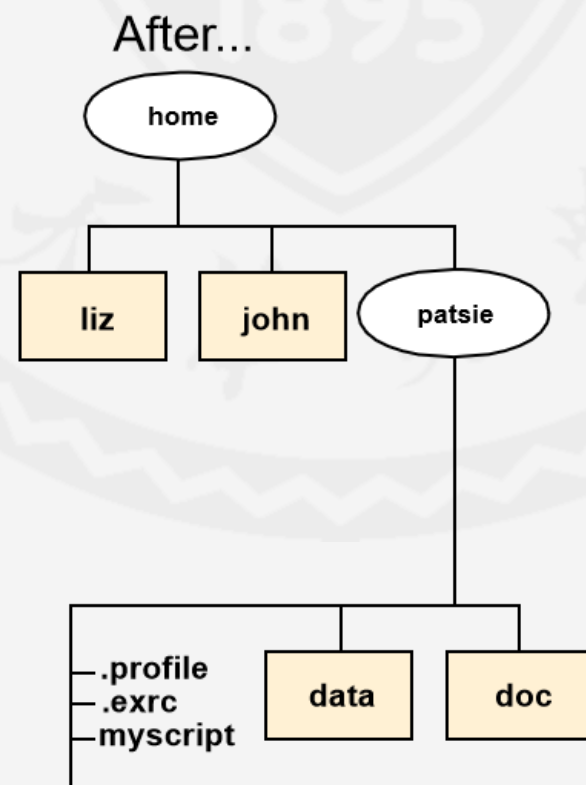
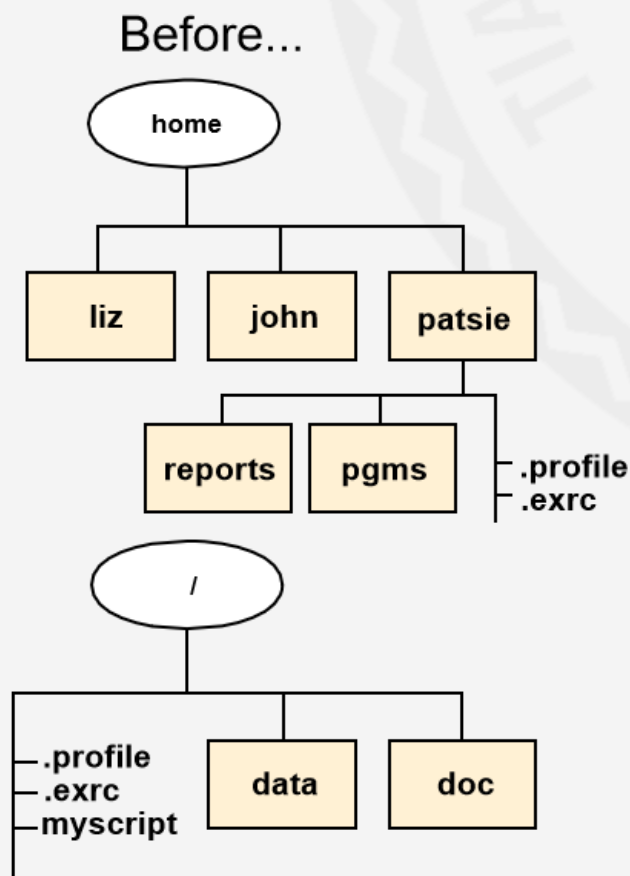




挂载文件系统

■ Windows采用盘符来标识不同的文件系统，而Linux中一个文件系统要想被访问，需要将其 **“挂载” (mount)** 到某个目录来实现，这个目录就是“挂载点”，解除关联关系的过程称之为“卸载” (umount)。

- (1) “挂载点” 目录应事先存在
- (2) 挂载点目录不可被其他进程使用
- (3) 挂载点下原有文件将被隐藏





课后阅读

- 《操作系统概念 OSC（第9版）》第10-11章。
- 《现代操作系统 MOS（第4版）》第4章